

Data structures

COMPARISON BASED SORTING ALGORITHMS

A solid orange horizontal bar spanning the width of the slide, located at the bottom.

Recitation Topics

- Recurrence complexity analysis - reminder
- Heap sort
- Merge sort

Question 1

Exercise :

Give an upper bound for the runtime of the following function.

```
func(n)
    if (n = 1 or n = 2)
        return 2n
    if (n mod 2 = 0)
        num = func(n/2)
        return num^2
    else
        return 2*func(n-1)
```

Question 1

Exercise :

Give an upper bound for the runtime of the following function.

```
func(n)
    if (n = 1 or n = 2)
        return 2n
    if (n mod 2 = 0)
        num = func(n/2)
        return num^2
    else
        return 2*func(n-1)
```

We solve the problem in two steps:

1. **Define the recurrence relation.**
Where are the recursive calls?
What are the base case/s?
2. **Solve the recurrence relation** to obtain a closed form for the running time as a function of n .

Question 1 - Solution

```
func(n)
  if (n = 1 or n = 2)
    return 2n
  if (n mod 2 = 0)
    num = func(n/2)
    return num^2
  else
    return 2*func(n-1)
```

- We begin by writing the recursive formula

$$\circ T(n) \leq \begin{cases} T\left(\frac{n}{2}\right) + c_3, & \text{if } n \text{ is even and } n > 2 \\ T(n-1) + c_4, & \text{if } n \text{ is odd and } n > 1 \end{cases}, \quad \begin{cases} T(1) = c_1 \\ T(2) = c_2 \end{cases}$$

- Let $c = 2 \max\{c_1, c_2, c_3, c_4\}$
- By applying the formula twice for odd n 's we may rewrite it as:

$$T(n) \leq T(\lfloor n/2 \rfloor) + c \quad \begin{cases} T(1) \leq c \\ T(2) \leq c \end{cases}$$

Question 1 - Solution

```
func(n)
  if (n = 1 or n = 2)
    return 2n
  if (n mod 2 = 0)
    num = func(n/2)
    return num^2
  else
    return 2*func(n-1)
```

Question 1 - Solution

```
func(n)
  if (n = 1 or n = 2)
    return 2n
  if (n mod 2 = 0)
    num = func(n/2)
    return num^2
  else
    return 2*func(n-1)
```

- $T(n) \leq T(\lfloor n/2 \rfloor) + c$

$$\begin{cases} T(1) \leq c \\ T(2) \leq c \end{cases}$$

$$T(n) \leq T(\lfloor n/2 \rfloor) + c \leq T(\lfloor n/4 \rfloor) + 2c \leq \dots \leq T(\lfloor n/2^k \rfloor) + k \cdot c$$

- This stops when $\left\lfloor \frac{n}{2^k} \right\rfloor \leq \frac{n}{2^k} \leq 2 \Rightarrow n \leq 2^{k+1} \Rightarrow \log_2 n - 1 \leq k$
- Therefore; after $\lfloor \log n \rfloor$ iterations we reach either $T(2)$ or $T(1)$

$$T(n) \leq c + \lfloor \log_2 n \rfloor \cdot c = O(\log n)$$

Question 1 - By Induction

```
func(n)
  if (n = 1 or n = 2)
    return 2n
  if (n mod 2 = 0)
    num = func(n/2)
    return num^2
  else
    return 2*func(n-1)
```

- **We need to prove that:** If $T(n) \leq T(\lfloor n/2 \rfloor) + c$, $\begin{cases} T(1) \leq c \\ T(2) \leq c \end{cases}$

Then $T(n) = O(\log n)$

- Enough to prove that for all $n \geq n_0 = 2$, $T(n) \leq c \cdot \log n$

- **Base Case:**

For $n = 2$: $T(2) \leq c = c \cdot \log_2 2$

Question 1 - By Induction

- **Induction Hypothesis:** we assume that for every $k < n$; $T(k) \leq c \cdot \log_2(k)$
- **Induction Step:** We prove for n ;

(1) When n is even then by the induction hypothesis;

$$\begin{aligned} T(n) &\leq T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + c = T\left(\frac{n}{2}\right) + c \leq c \log_2\left(\frac{n}{2}\right) + c = \\ &= c (\log_2(n) - \log_2(2)) + c = c \log_2(n) - c + c = c \log_2(n) \end{aligned}$$

Question 1 - By Induction

- **Induction Hypothesis:** we assume that for every $k < n$; $T(k) \leq c \cdot \log_2(k)$
- **Induction Step:** We prove for n ;

(2) When n is odd;

$$\begin{aligned} T(n) &\leq T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + c = T\left(\frac{n-1}{2}\right) + c \leq \\ c \cdot (\log_2(n-1) - \log_2 2) + c &\leq c \cdot \log_2(n-1) \leq c \log_2(n) \end{aligned}$$

Sorting algorithm visualizations

- Sorting algorithms simulations

- <https://www.toptal.com/developers/sorting-algorithms>

- <https://youtu.be/kPRA0W1kECg?si=Tk1il7srqipny9vH&t=89>

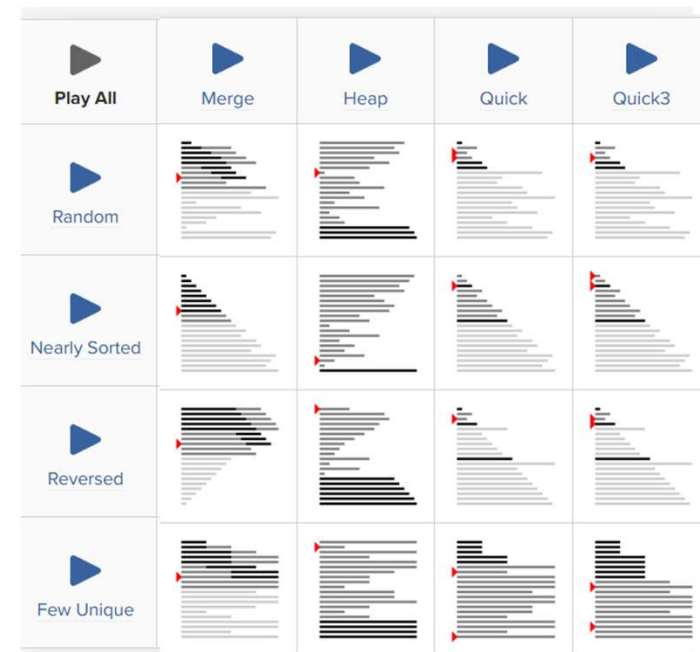
- AlgoRythmics

- Merge-sort :

- https://www.youtube.com/watch?v=XaqR3G_NVoo&list=PLcX11VWS1PdA4dSPip8-1JfKxFa32X53y&index=3&ab_channel=AlgoRythmics

- Heap-sort:

- https://www.youtube.com/watch?v=Xw2D9aJRBY4&list=PLcX11VWS1PdA4dSPip8-1JfKxFa32X53y&ab_channel=AlgoRythmics



Sorting algorithm visualizations

- Sorting algorithms simulations

- <https://www.toptal.com/developers/sorting-algorithms>

- <https://youtu.be/kPRA0W1kECg?si=Tk1il7srqipny9vH&t=89>

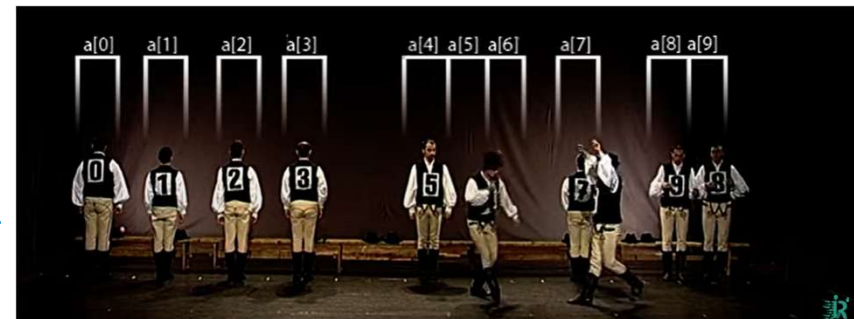
- AlgoRythmics

- Merge-sort :

- https://www.youtube.com/watch?v=XaqR3G_NVoo&list=PLcX11VWS1PdA4dSPip8-1JfKxF32X53y&index=3&ab_channel=AlgoRythmics

- Heap-sort:

- https://www.youtube.com/watch?v=Xw2D9aJRBY4&list=PLcX11VWS1PdA4dSPip8-1JfKxF32X53y&ab_channel=AlgoRythmics



Heapsort High-Level Idea

Build-heap: Our first step is to transform the input list into a heap (a.k.a. "heapify" it).

- Reminder - The tree corresponding to the heap is represented by an array where $A[1]$ is the root and the children of $A[i]$ are $A[2i]$ and $A[2i + 1]$.

Repeated DeleteMax: Each time we remove an element from the heap, it's the largest item in the underlying list of remaining items. So, in sorted order, it is moved to the end of the array.

Heapsort Pseudocode

HeapSort(A)

 BuildMaxHeap(A)

 n=length(A)

 For s=n downto 2

 x=A[1]

 PercDown(1, A[s], s-1, A)

 A[h]=x

Heapsort Complexity

Time complexity:

- **Average case:** $\theta(n \log n)$
- **Worst case:** $\theta(n \log(n))$
- **Best case:** $\theta(n)$

Space complexity:

- $\theta(1)$ additional space - way better than merge sort's $\theta(n)$ overhead.

Question 2

Suppose we need a modified Heapsort that sorts elements in *descending order*.
What changes should we make to the pseudo-code of Heapsort?

Question 2 - Solution

Two straightforward solutions:

HeapSort(A)

BuildMinHeap(A)

n=length(A)

For h=n downto 2

 x=A[1]

 PercDown(1, A[h], h-1, A)

 A[h]=x

HeapSort(A)

BuildMaxHeap(A)

n=length(A)

For h=n downto 2

 x=A[1]

 PercDown(1, A[h], h-1, A)

 A[h]=x

ReverseArray(A)

Pros and cons?

Heapsort Summary

Strengths:

- **Fast:** Heap sort runs in $\theta(n\log(n))$ time, which scales well as n grows. Unlike bubble sort, there's no worst-case $O(n^2)$ complexity.
- **Space efficient:** Heap sort is an in-place sorting algorithm (takes at most $O(1)$ additional space). That's way better than merge sort's $O(n)$ overhead.

Weaknesses:

- **Slow in practice.** While the asymptotic complexity of heapsort is faster than quicksort in the worst case, on the average case heapsort is often slower in practice.

Merge Sort High-Level Idea

Divide the n -element sequence into 2 subsequences, each with $\frac{n}{2}$ elements

Conquer: Sort the two subsequences recursively **using recursive merge sort**.

Merge the two sorted subsequences to produce the sorted answer

The base case: a sequence with 1 element is sorted.

Recursive Merge Sort Pseudocode

```
Mergesort(A, l, r)
if l < r then
    q ←  $\lfloor (l+r)/2 \rfloor$ 
    Mergesort(A, l, q)
    Mergesort(A, q+1, r)
    Merge(A, l, q, r)
```

A: the input and output array
A[l...r]: subarray
l, r: left (first) and right (last)
indexes of the in A

Merge(A, p, q, r) is a procedure that merges the 2 subarrays A[p,...,q] and A[q+1,...,r].

MERGE(A, p, q, r)

```
1   $n_1 = q - p + 1$ 
2   $n_2 = r - q$ 
3  let  $L[1..n_1 + 1]$  and  $R[1..n_2 + 1]$  be new arrays
4  for  $i = 1$  to  $n_1$ 
5       $L[i] = A[p + i - 1]$ 
6  for  $j = 1$  to  $n_2$ 
7       $R[j] = A[q + j]$ 
8   $L[n_1 + 1] = \infty$ 
9   $R[n_2 + 1] = \infty$ 
10  $i = 1$ 
11  $j = 1$ 
12 for  $k = p$  to  $r$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 
```

Merge-Sort - Example

```
Mergesort(A, l, r)
if l < r then
    q ← ⌊(l+r)/2⌋
    Mergesort(A, l, q)
    Mergesort(A, q+1, r)
    Merge(A, l, q, r)
```

1	5	4	8	10	2	6	9	12	11	3	7
---	---	---	---	----	---	---	---	----	----	---	---

1	5	4	8	10	2
---	---	---	---	----	---

6	9	12	11	3	7
---	---	----	----	---	---

1	5	4
---	---	---

8	10	2
---	----	---

6	9	12
---	---	----

11	3	7
----	---	---

1	5	4
---	---	---

8	10	2
---	----	---

6	9	12
---	---	----

11	3	7
----	---	---

1	4	5
---	---	---

8	2	10
---	---	----

6	9	12
---	---	----

11	3	7
----	---	---

1	4	5
---	---	---

2	8	10
---	---	----

6	9	12
---	---	----

3	7	11
---	---	----

1	2	4	5	8	10
---	---	---	---	---	----

3	6	7	9	11	12
---	---	---	---	----	----

1	2	3	4	5	6	7	8	9	10	11	12
---	---	---	---	---	---	---	---	---	----	----	----

Merge Sort Complexity

Run Time complexity:

- **Average case:** $\theta(n \log n)$
- **Worst case:** $\theta(n \log(n))$, $T(n) \leq 2T\left(\frac{n}{2}\right) + c_1 n \Rightarrow T(n) \in \theta(n \log n)$
- **Best case:** $\theta(n \log(n))$

Space complexity:

- $\theta(n)$

Merge Sort Summary

Strengths:

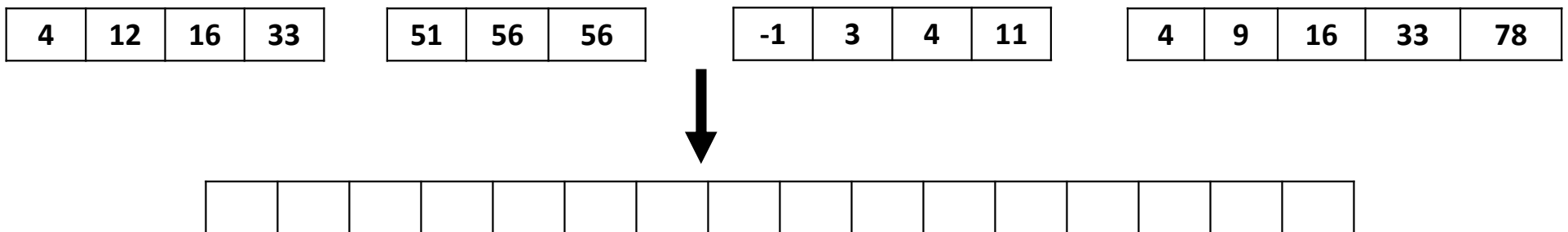
- **Fast.** Merge sort runs in $O(n \log(n))$, which scales well as n grows.
- **Parallelizable.** Merge sort breaks the input into two chunks, each of which can be sorted at the same time in parallel.

Weaknesses:

- **Space.** Merge sort takes up $O(n)$ extra space, including $O(\log n)$ space for the recursive call stack.
 - A lot of copy operations, hence larger constants.
- 

Question 3

Design an efficient algorithm to merge k sorted arrays, where n is the total number of elements in all the input arrays.



Question 3 - Solution

- The straightforward solution is to pick the smallest of the top elements in each array, repeatedly. This takes $k - 1$ comparisons per element, in total $O(k \cdot n)$
- Another naive solution is to create an output array and one by one copy all k arrays to it. Finally, sort the output array. This approach takes $O(n \log n)$.
- An efficient way to merge k sorted arrays with a total of n elements in $O(n \log k)$ time is by using a *min-heap* data structure.

Question 3 - Solution (algorithm)

A high-level description of the algorithm:

1. Initialize a *min-heap* with each array first element (as key), and the array index and element index in the array (as data).
2. While the *min-heap* is not empty:
 - a. Extract the min element (with its array and element index).
 - b. Add the extracted element to the merged output array.
 - c. If more elements in the source array exist, insert the next element into the *min-heap*.
3. When the *min-heap* is empty, the output array is sorted and contains all the elements of the original arrays.



Question 3 - Solution (time analysis)

Analysis: It takes $O(k)$ to build the heap; for every element, it takes $O(\log k)$ to DeleteMin and $O(\log k)$ to insert the next one from the same array.

In total it takes $O(k + n \log k) = O(n \log k)$.

Question 4

Consider the following algorithm, which gets an array A of size n :

```
weird-sort(A):  
  if (n > 10)  
    sort recursively the first 2/3 elements of A (from 1 to  $\lfloor \frac{2n}{3} \rfloor$ )  
    sort recursively the last 2/3 elements of A (from  $\lfloor \frac{n}{3} \rfloor$  to  $n$ )  
    sort recursively the first 2/3 elements of A (from 1 to  $\lfloor \frac{2n}{3} \rfloor$ )  
  else use bubbleSort
```

Decide whether the algorithm sorts A .

If not, give a counter example. If yes, explain why and analyze its runtime.

Question 4 - Example

weird-sort(A):

sort recursively the first $\frac{2}{3}$ elements of A (from 1 to $\lfloor \frac{2n}{3} \rfloor$)

sort recursively the last $\frac{2}{3}$ elements of A (from $\lfloor \frac{n}{3} \rfloor$ to n)

sort recursively the first $\frac{2}{3}$ elements of A (from 1 to $\lfloor \frac{2n}{3} \rfloor$)

7	1	6	8	10	5	11	9	3	4	2	12
---	---	---	---	----	---	----	---	---	---	---	----

After step one

1	5	6	7	8	9	10	11	3	4	2	12
---	---	---	---	---	---	----	----	---	---	---	----

After step two

1	5	6	7	2	3	4	8	9	10	11	12
---	---	---	---	---	---	---	---	---	----	----	----

After step three

1	2	3	4	5	6	7	8	9	10	11	12
---	---	---	---	---	---	---	---	---	----	----	----

Question 4 - Solution

```
weird-sort(A):  
    if (n > 10)  
        sort recursively the first 2/3 elements of A (from 1 to  $\lfloor \frac{2n}{3} \rfloor$ )  
        sort recursively the last 2/3 elements of A (from  $\lfloor \frac{n}{3} \rfloor$  to n)  
        sort recursively the first 2/3 elements of A (from 1 to  $\lfloor \frac{2n}{3} \rfloor$ )  
    else use bubbleSort
```

Solution:

Note that after the first two steps, it is necessarily true that the $\frac{n}{3}$ largest elements are already in their place. (The first step assured that they will be in the last $\frac{2n}{3}$ locations and the second step actually put them in their place).

The third step sorts the remaining elements.

Question 4 - Solution

```
weird-sort(A):  
    if (n > 10)  
        sort recursively the first 2/3 elements of A (from 1 to  $\lceil \frac{2n}{3} \rceil$ )  
        sort recursively the last 2/3 elements of A (from  $\lfloor \frac{n}{3} \rfloor$  to n)  
        sort recursively the first 2/3 elements of A (from 1 to  $\lceil \frac{2n}{3} \rceil$ )  
    else use BubbleSort
```

Runtime:

The runtime is given by the formula

$$T(n) = \begin{cases} 3T\left(\frac{2n}{3}\right) & \text{if } n > 10 \\ \theta(1) & \text{else} \end{cases}$$

Question 4 - Solution

$$T(n) = \begin{cases} 3T\left(\frac{2n}{3}\right) , & \text{if } n > 10 \\ \theta(1) & , \quad \text{else} \end{cases}$$

Question 4 - Solution

$$T(n) = 3T\left(\frac{2n}{3}\right) \quad \text{if } n > 10$$

$$T(n) \leq c_1 \quad \text{if } n \leq 10$$

$$T(n) = 3T\left(\frac{2n}{3}\right) = 3 \cdot 3 \cdot T\left(\left(\frac{2}{3}\right)^2 n\right) = \dots = 3^k \cdot T\left(\left(\frac{2}{3}\right)^k n\right)$$

Our stopping condition is met when;

$$\left(\frac{2}{3}\right)^k \cdot n < 10 \quad \Leftrightarrow \quad \frac{n}{10} < \left(\frac{3}{2}\right)^k \quad \Leftrightarrow \quad k > \log_{\frac{3}{2}} \left(\frac{n}{10}\right)$$

$$\log_{\frac{3}{2}}(x) = \frac{\log_3 x}{\log_3 \frac{3}{2}} = \log_3(x) \cdot \log_{\left(\frac{3}{2}\right)} 3$$

Therefore, for $k = \left\lceil \log_{\frac{3}{2}} \left(\frac{n}{10}\right) \right\rceil$:

$$T(n) \leq c_1 \cdot 3^k = c_1 \cdot 3^{\left\lceil \log_{\left(\frac{3}{2}\right)} \left(\frac{n}{10}\right) \right\rceil} \leq c_1 \cdot 3^{1 + \log_{\left(\frac{3}{2}\right)} \left(\frac{n}{10}\right)} = c_1 \cdot 3^{(1 - \log_{\left(\frac{3}{2}\right)} 10) + \log_{\left(\frac{3}{2}\right)} n} \quad (*) \text{ For some } C \geq c_1 \cdot 3^{1 - \log_{\left(\frac{3}{2}\right)} 10}$$

$$\leq C \cdot 3^{\log_{\left(\frac{3}{2}\right)} n} = C \cdot 3^{\log_{\left(\frac{3}{2}\right)} 3 \cdot \log_3(n)} = C n^{\log_{\left(\frac{3}{2}\right)} 3} = O\left(n^{\log_{\left(\frac{3}{2}\right)} 3}\right) \subset O(n^{2.71})$$

$$(**) \log_{\frac{3}{2}}(3) \cong 2.709$$

Question 4 - Solution

$$T(n) = 3T\left(\frac{2n}{3}\right) \quad \text{if } n > 10$$

$$T(n) \geq c_2 \quad \text{if } n \leq 10$$

$$T(n) = 3T\left(\frac{2n}{3}\right) = 3 \cdot 3 \cdot T\left(\frac{2n}{3}\right) = \dots = 3^k \cdot T\left(\left(\frac{2}{3}\right)^k n\right)$$

Our stopping condition is met when;

$$\left(\frac{2}{3}\right)^k n < 10 \quad \Leftrightarrow \quad \frac{n}{10} < \left(\frac{3}{2}\right)^k \Leftrightarrow \quad k > \log_{\frac{3}{2}} \left(\frac{n}{10}\right)$$

$$\log_{\frac{3}{2}}(x) = \frac{\log_3 x}{\log_3 \left(\frac{3}{2}\right)} = \log_3(x) \cdot \log_{\left(\frac{3}{2}\right)} 3$$

Therefore, for $k = \left\lceil \log_{\frac{3}{2}} \left(\frac{n}{10}\right) \right\rceil$:

$$\begin{aligned} T(n) &\geq c_2 \cdot 3^k = c_2 \cdot 3^{\left\lceil \log_{\left(\frac{3}{2}\right)} \left(\frac{n}{10}\right) \right\rceil} \geq c_2 \cdot 3^{\log_{\left(\frac{3}{2}\right)} \left(\frac{n}{10}\right)} = c_2 \cdot 3^{-\log_{\left(\frac{3}{2}\right)} 10} \cdot 3^{\log_{\left(\frac{3}{2}\right)} n} \quad (*) \text{ For some } C' \leq c_2 \cdot 3^{-\log_{\left(\frac{3}{2}\right)} 10} \\ &\geq C' \cdot 3^{\log_{\left(\frac{3}{2}\right)} 3 \cdot \log_3(n)} = C' n^{\log_{\left(\frac{3}{2}\right)} 3} = \Omega\left(n^{\log_{\left(\frac{3}{2}\right)} 3}\right) \subset \Omega(n^{2.709}) \end{aligned}$$

$$(**) \log_{\frac{3}{2}}(3) \cong 2.709$$